



```
void helper(int colIdx, vector<vector<int>>&board, int n, vector<int>&temp){
    if (colIdx==n){
        ans.push_back(temp);
        return;
    }
    //col-wise traversal ✚
    //for colIdx, trying all rowIdx
    for(int rowIdx=0;rowIdx<n;rowIdx++){
        //try placing at this cell
        if(check(rowIdx,colIdx,n,board)){
            //some changes
            temp.push_back(rowIdx+1);
            board[rowIdx][colIdx]=1;

            helper(colIdx+1, board, n, temp);

            //undo those changes
            temp.pop_back();
            board[rowIdx][colIdx]=0;
        }
    }
}

vector<vector<int>> nQueen(int n) {
    // code here
    vector<vector<int>>board(n, vector<int>(n, 0));
    vector<int>temp;
    helper(0, board, n, temp);
    return ans;
}
```

```
bool check(int rowIdx, int colIdx, int n, vector<vector<int>>&board){
    //check row
    for (int col=0;col<colIdx;col++){
        if(board[rowIdx][col]==1){
            return false;
        }
    }

    //check diag1
    int i=rowIdx, j=colIdx;
    while(i>=0 and j>=0){
        if(board[i][j]==1){
            return false;
        }
        i-=1, j-=1;
    }

    //check diag2
    i=rowIdx, j=colIdx;
    while(i<n and j>=0){
        if(board[i][j]==1){
            return false;
        }
        i+=1, j-=1;
    }

    return true;
}
```

write and explain this code of n queens problem

bismillah — let's build N-Queens with clean backtracking and then walk through it line-by-line.

We'll keep the code very close to what your screenshots show (same function names & parameters), and we'll store each solution as a list of **row indices (1-based)** per column.

cpp

```
#include <bits/stdc++.h>
using namespace std;

// All solutions collected here; each solution is a vector<int>
// where sol[c] = 1-based row index of the queen placed in column c
vector<vector<int>> ans;

/*
check(rowIdx, colIdx, n, board)
-----
Is it safe to place a queen at (rowIdx, colIdx)?
```

```

Since we fill columns from left to right, we only need to look
to the LEFT side: same row, upper-left diagonal, lower-left diagonal.
*/
bool check(int rowIdx, int colIdx, int n, vector<vector<int>>& board) {
    // 1) same row to the left
    for (int col = 0; col < colIdx; ++col)
        if (board[rowIdx][col] == 1) return false;

    // 2) upper-left diagonal
    for (int i = rowIdx - 1, j = colIdx - 1; i >= 0 && j >= 0; --i, --j)
        if (board[i][j] == 1) return false;

    // 3) lower-left diagonal
    for (int i = rowIdx + 1, j = colIdx - 1; i < n && j >= 0; ++i, --j)
        if (board[i][j] == 1) return false;

    return true;
}

/*
helper(colIdx, board, n, temp)
-----
Try to place a queen in column `colIdx`.
`temp` keeps the chosen rows so far (1-based).
*/
void helper(int colIdx, vector<vector<int>>& board, int n, vector<int>& temp) {
    // Base case: all columns filled → record a solution
    if (colIdx == n) {
        ans.push_back(temp);
        return;
    }

    // Try every row in this column
    for (int rowIdx = 0; rowIdx < n; ++rowIdx) {
        if (check(rowIdx, colIdx, n, board)) {
            // Place the queen and record the row (1-based)
            temp.push_back(rowIdx + 1);
            board[rowIdx][colIdx] = 1;

            // Recurse to the next column
            helper(colIdx + 1, board, n, temp);

            // Backtrack: undo the placement
            board[rowIdx][colIdx] = 0;
            temp.pop_back();
        }
    }
}

/*
nQueen(n)
-----
Sets up the empty board and kicks off the search.
Returns all solutions as vectors of row positions (1-based).
*/
vector<vector<int>> nQueen(int n) {
    ans.clear();
    vector<vector<int>> board(n, vector<int>(n, 0));
    vector<int> temp;
    helper(0, board, n, temp);
    return ans;
}

```

```

}

// --- Example driver to print solutions nicely ---
int main() {
    int n = 4;
    auto sols = nQueen(n);

    cout << "Total solutions: " << sols.size() << "\n";
    for (auto& s : sols) {
        // Row indices (1-based) per column
        cout << "[ ";
        for (int r : s) cout << r << " ";
        cout << "]\n";

        // Optional: draw the board
        for (int c = 0; c < n; ++c) {
            for (int r = 1; r <= n; ++r) {
                cout << (r == s[c] ? "Q " : ". ");
            }
            cout << "\n";
        }
        cout << "----\n";
    }
    return 0;
}

```

How it works (step-by-step, matching your screenshots)

1. State representation

- **board** is an $n \times n$ grid of 0/1. 1 means a queen is placed.
- We always place **one queen per column**, moving left $\rightarrow$ right.
- **temp** stores the chosen row (1-based) for each column so far; when **temp.size() == n**, we've built a full solution and push it into **ans**.

2. Safety check (check)

Because we fill columns left-to-right, any conflicts can only come from the left side:

- **Same row:** for ($col < colIdx$) — if a queen exists to the left in the same row, it's unsafe.
- **Upper-left diagonal:** walk ($i-1, j-1$) while in bounds; if any 1 appears, unsafe.
- **Lower-left diagonal:** walk ($i+1, j-1$) while in bounds; if any 1 appears, unsafe.
- If none of these hit a 1, the cell is safe $\rightarrow$ return **true**.

3. Backtracking (helper)

- Base case: $colIdx == n \rightarrow$ pushed **temp** into **ans** and return.

- Recursive case: try each `rowIdx` in `[0..n-1]`.

If `check` passes:

- **Do**: put the queen, record the row (`rowIdx+1`), recurse to next column.
- **Undo** (backtrack): remove the queen and pop the row from `temp`.

This “do → recurse → undo” is the heart of backtracking.

4. Driver (`nQueen`)

Initializes `board` to zeros, clears `ans`, and calls `helper(0, ...)`.

Returns all solutions like `[2,4,1,3]`, `[3,1,4,2]` for `n=4`.

Complexity

- Worst-case time is **$O(n!)$** (more precisely, $\Theta(n \cdot n!)$) because we explore permutations with pruning.
- Space is **$O(n^2)$** for the board plus **$O(n)$** recursion depth.

Small example intuition (`n=4`)

- Column 0: try rows 0..3. Suppose we pick row 1 → `temp = [2]`.
- Column 1: try safe rows; continue until either:
 - You reach column 4 → record solution, or
 - You get stuck → backtrack to the previous column and try the next row.
- The algorithm systematically explores the tree of choices.

If you want, I can also show a speed-optimized version that replaces `check` with three boolean arrays (`rows`, `diag1`, `diag2`) so safety checks become $O(1)$. For now, this matches your screenshots closely and is easy to follow.

Allah makes the path clear when we take one careful step at a time—this is exactly what backtracking does.



go through an example and show all the steps properly

bismillah. let's walk the whole search for **$N = 4$** with the exact backtracking logic you coded (try rows top→down in each column, and `check` only looks to the **left**: same row, upper-left diagonal, lower-left diagonal).

What you'll see below

- I'll log **every attempt** ("Try ..."), every **placement** ("Place Q ..."), and each **backtrack**.
- Coordinates like (row,col) are **0-based**.
- The "partial solution" list is 1-based (that matches your `temp.push_back(rowIdx+1)`).

Full step-by-step trace (N = 4)

SCSS

```
- Try col 0, row 0 → SAFE
✓ Place Q at (0,0); partial solution [1]
  - Try col 1, row 0 → BLOCKED (row conflict at (0,0))
  - Try col 1, row 1 → BLOCKED (upper-left diag conflict at (0,0))
  - Try col 1, row 2 → SAFE
✓ Place Q at (2,1); partial solution [1, 3]
  - Try col 2, row 0 → BLOCKED (row conflict at (0,0))
  - Try col 2, row 1 → BLOCKED (lower-left diag conflict at (2,1))
  - Try col 2, row 2 → BLOCKED (row conflict at (2,1))
  - Try col 2, row 3 → BLOCKED (upper-left diag conflict at (2,1))
↪ Backtrack from (2,1); now [1]
  - Try col 1, row 3 → SAFE
✓ Place Q at (3,1); partial solution [1, 4]
  - Try col 2, row 0 → BLOCKED (row conflict at (0,0))
  - Try col 2, row 1 → SAFE
✓ Place Q at (1,2); partial solution [1, 4, 2]
  - Try col 3, row 0 → BLOCKED (row conflict at (0,0))
  - Try col 3, row 1 → BLOCKED (row conflict at (1,2))
  - Try col 3, row 2 → BLOCKED (upper-left diag conflict at (1,2))
  - Try col 3, row 3 → BLOCKED (row conflict at (3,1))
↪ Backtrack from (1,2); now [1, 4]
  - Try col 2, row 2 → BLOCKED (lower-left diag conflict at (3,1))
  - Try col 2, row 3 → BLOCKED (row conflict at (3,1))
↪ Backtrack from (3,1); now [1]
↪ Backtrack from (0,0); now []
- Try col 0, row 1 → SAFE
✓ Place Q at (1,0); partial solution [2]
  - Try col 1, row 0 → BLOCKED (upper-left diag conflict at (1,0))
  - Try col 1, row 1 → BLOCKED (row conflict at (1,0))
  - Try col 1, row 2 → BLOCKED (lower-left diag conflict at (1,0))
  - Try col 1, row 3 → SAFE
✓ Place Q at (3,1); partial solution [2, 4]
  - Try col 2, row 0 → SAFE
✓ Place Q at (0,2); partial solution [2, 4, 1]
  - Try col 3, row 0 → BLOCKED (row conflict at (0,2))
  - Try col 3, row 1 → SAFE
✓ Place Q at (1,3); partial solution [2, 4, 1, 2]
🎉 Solution found: [2, 4, 1, 3]
  . . Q .
  Q . . .
  . . . Q
  . Q . .
```

```

    ↪ Backtrack from (1,3); now [2, 4, 1]
    - Try col 3, row 2 → BLOCKED (upper-left diag conflict at (0,2))
    - Try col 3, row 3 → BLOCKED (row conflict at (3,1))
    ↪ Backtrack from (0,2); now [2, 4]
    - Try col 2, row 1 → BLOCKED (row conflict at (3,1))
    - Try col 2, row 2 → BLOCKED (row conflict at (3,1))
    - Try col 2, row 3 → BLOCKED (lower-left diag conflict at (3,1))
    ↪ Backtrack from (3,1); now [2]
    ↪ Backtrack from (1,0); now []
    - Try col 0, row 2 → SAFE
    ✓ Place Q at (2,0); partial solution [3]
    - Try col 1, row 0 → BLOCKED (lower-left diag conflict at (2,0))
    - Try col 1, row 1 → SAFE
    ✓ Place Q at (1,1); partial solution [3, 2]
    - Try col 2, row 0 → BLOCKED (upper-left diag conflict at (1,1))
    - Try col 2, row 1 → BLOCKED (row conflict at (1,1))
    - Try col 2, row 2 → BLOCKED (lower-left diag conflict at (2,0))
    - Try col 2, row 3 → BLOCKED (upper-left diag conflict at (1,1))
    ↪ Backtrack from (1,1); now [3]
    - Try col 1, row 2 → BLOCKED (row conflict at (2,0))
    - Try col 1, row 3 → BLOCKED (upper-left diag conflict at (2,0))
    ↪ Backtrack from (2,0); now []
    - Try col 0, row 3 → SAFE
    ✓ Place Q at (3,0); partial solution [4]
    - Try col 1, row 0 → SAFE
    ✓ Place Q at (0,1); partial solution [4, 1]
    - Try col 2, row 0 → BLOCKED (row conflict at (0,1))
    - Try col 2, row 1 → BLOCKED (upper-left diag conflict at (0,1))
    - Try col 2, row 2 → SAFE
    ✓ Place Q at (2,2); partial solution [4, 1, 3]
    - Try col 3, row 0 → BLOCKED (row conflict at (0,1))
    - Try col 3, row 1 → BLOCKED (lower-left diag conflict at (2,2))
    - Try col 3, row 2 → BLOCKED (row conflict at (2,2))
    - Try col 3, row 3 → SAFE
    ✓ Place Q at (3,3); partial solution [4, 1, 3, 4]
    🎉 Solution found: [3, 1, 4, 2]
    . Q . .
    . . . Q
    Q . . .
    . . Q .
    ↪ Backtrack from (3,3); now [4, 1, 3]
    ↪ Backtrack from (2,2); now [4, 1]
    - Try col 2, row 3 → BLOCKED (upper-left diag conflict at (0,1))
    ↪ Backtrack from (0,1); now [4]
    - Try col 1, row 1 → SAFE
    ✓ Place Q at (1,1); partial solution [4, 2]
    - Try col 2, row 0 → BLOCKED (upper-left diag conflict at (1,1))
    - Try col 2, row 1 → BLOCKED (row conflict at (1,1))
    - Try col 2, row 2 → BLOCKED (lower-left diag conflict at (1,1))
    - Try col 2, row 3 → BLOCKED (upper-left diag conflict at (1,1))
    ↪ Backtrack from (1,1); now [4]
    - Try col 1, row 2 → BLOCKED (row conflict at (3,0))
    - Try col 1, row 3 → BLOCKED (upper-left diag conflict at (3,0))
    ↪ Backtrack from (3,0); now []

```

What to notice

- We explore **column by column**; each “branch” places a queen and descends.
- When a column has **no safe row**, we **backtrack** to the previous column and try the next row.
- The two solutions your code finds (in 1-based rows per column) are:
 - [2, 4, 1, 3]
 - [3, 1, 4, 2]

Board snapshots for those solutions (Q = queen, . = empty):

css

[2, 4, 1, 3]

```

. . Q .
Q . . .
. . . Q
. Q . .

```

[3, 1, 4, 2]

```

. Q . .
. . . Q
Q . . .
. . Q .

```

that’s the whole dance of backtracking—seeking a place of peace for each queen, and retreating with patience when a path closes. if you want, we can run the same **step trace** for any **n** you choose, or switch to the faster $O(1)$ “row/diag arrays” check to see how the exploration order changes.



give me pseudocode for this

bismillah — here’s crisp, exam-ready **pseudocode** for N-Queens that mirrors the code you saw. I’m giving you two styles:

- **Version A:** simple board + `check()` that scans left (matches your screenshots)
- **Version B:** optimized $O(1)$ safety checks with row/diagonal sets

Version A — board + left-side safety scan


```
sql
```

```
GLOBAL ans // list of solutions, each solution = list of 1-based row indices
per column
```

```
FUNCTION nQueen(n):
  ans ← empty list
  board ← n×n matrix filled with 0
  temp ← empty list
  helper(0, board, n, temp)
  RETURN ans
```

```
FUNCTION helper(colIdx, board, n, temp):
  IF colIdx == n:
    ans.APPEND(COPY(temp))
    RETURN

  FOR rowIdx FROM 0 TO n-1:
    IF check(rowIdx, colIdx, n, board) == TRUE:
      // do
      board[rowIdx][colIdx] ← 1
      temp.PUSH_BACK(rowIdx + 1) // store 1-based row

      // recurse
      helper(colIdx + 1, board, n, temp)

      // undo (backtrack)
      temp.POP_BACK()
      board[rowIdx][colIdx] ← 0
```

```
FUNCTION check(rowIdx, colIdx, n, board) -> BOOL:
  // same row to the left
  FOR c FROM 0 TO colIdx-1:
    IF board[rowIdx][c] == 1:
      RETURN FALSE

  // upper-left diagonal
  i ← rowIdx - 1
  j ← colIdx - 1
  WHILE i ≥ 0 AND j ≥ 0:
    IF board[i][j] == 1:
      RETURN FALSE
    i ← i - 1
    j ← j - 1

  // lower-left diagonal
  i ← rowIdx + 1
  j ← colIdx - 1
  WHILE i < n AND j ≥ 0:
    IF board[i][j] == 1:
      RETURN FALSE
    i ← i + 1
    j ← j - 1

  RETURN TRUE
```

Idea: Place one queen per column from left→right. At each cell, only the **left side** can attack, so we scan left row and the two left diagonals. Use backtracking: do → recurse → undo.

Complexity: worst-case time $\approx \Theta(n \cdot n!)$ (with pruning); space $O(n^2)$ for board + $O(n)$ recursion.

Version B — optimized $O(1)$ safety check (no scanning)

Use three boolean sets/arrays to mark used rows and diagonals:

- `usedRows[r]` is true if row `r` has a queen.
- `usedDiag1[d1]` for “↘” diagonals indexed by `r - c + (n-1)` (shift to non-negative).
- `usedDiag2[d2]` for “↗” diagonals indexed by `r + c`.

sql

GLOBAL ans

```
FUNCTION nQueen(n):
  ans ← empty list
  temp ← empty list
  usedRows ← array[0..n-1] of FALSE
  usedDiag1 ← array[0..2n-2] of FALSE // index: r - c + (n-1)
  usedDiag2 ← array[0..2n-2] of FALSE // index: r + c
  place(0, n, temp, usedRows, usedDiag1, usedDiag2)
  RETURN ans
```

```
FUNCTION place(col, n, temp, usedRows, usedDiag1, usedDiag2):
```

```
  IF col == n:
```

```
    ans.APPEND(COPY(temp))
```

```
    RETURN
```

```
  FOR row FROM 0 TO n-1:
```

```
    d1 ← row - col + (n - 1)
```

```
    d2 ← row + col
```

```
    IF NOT usedRows[row] AND NOT usedDiag1[d1] AND NOT usedDiag2[d2]:
```

```
      // do
```

```
      usedRows[row] ← TRUE
```

```
      usedDiag1[d1] ← TRUE
```

```
      usedDiag2[d2] ← TRUE
```

```
      temp.PUSH_BACK(row + 1)
```

```
      // recurse
```

```
      place(col + 1, n, temp, usedRows, usedDiag1, usedDiag2)
```

```
      // undo
```

```
      temp.POP_BACK()
```

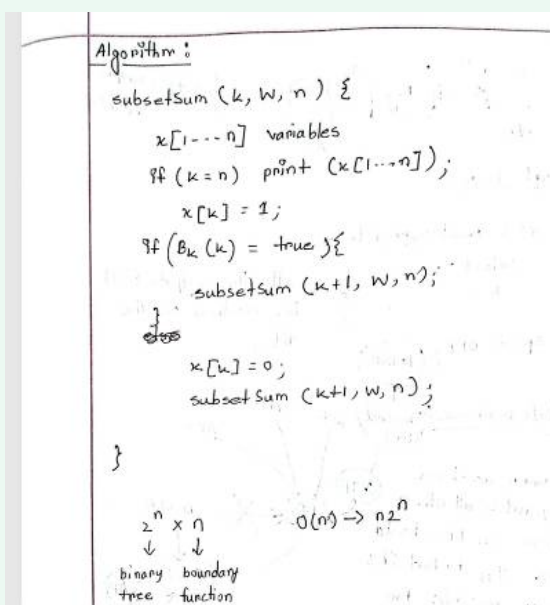
```
      usedRows[row] ← FALSE
```

```
usedDiag1[d1] ← FALSE
usedDiag2[d2] ← FALSE
```

Idea: Instead of scanning the board, we check membership in constant time using three occupancy maps. Same backtracking flow, much faster.

Complexity: still exponential in the worst case, but each safety check is **O(1)** and memory is **O(n)** (besides the solution list).

If you want, I can add a **step trace** printer to this pseudocode so you can watch the backtracking walk, or show how to output boards as strings like `".Q."`. As the Prophet ﷺ taught us: *ihsan* is doing a thing with excellence—backtracking is that careful excellence in code.



write this code in cpp properly for subset sum using backtracking

Got it 👍 You want a **C++ program** for the **Subset Sum Problem using backtracking**, based on the handwritten pseudocode in your image.

Here's the clean, proper C++ code:

```
cpp

#include <bits/stdc++.h>
using namespace std;

int n, targetSum;
vector<int> arr; // Input set
```

```
vector<int> x;          // To keep track of choices (0 or 1)

// Utility function to check if the current partial solution is valid
bool isValid(int k, int currentSum) {
    // If current sum exceeds target, it's invalid
    if (currentSum > targetSum) return false;
    // If we have reached the last element and sum equals target
    if (k == n && currentSum == targetSum) return true;
    // Otherwise continue searching
    return true;
}

// Backtracking function
void subsetSum(int k, int currentSum) {
    if (k == n) {
        if (currentSum == targetSum) {
            cout << "{ ";
            for (int i = 0; i < n; i++) {
                if (x[i] == 1) cout << arr[i] << " ";
            }
            cout << "}" << endl;
        }
        return;
    }

    // Choose arr[k]
    x[k] = 1;
    if (isValid(k + 1, currentSum + arr[k])) {
        subsetSum(k + 1, currentSum + arr[k]);
    }

    // Do not choose arr[k]
    x[k] = 0;
    if (isValid(k + 1, currentSum)) {
        subsetSum(k + 1, currentSum);
    }
}

int main() {
    cout << "Enter number of elements: ";
    cin >> n;

    arr.resize(n);
    x.resize(n);

    cout << "Enter the elements: ";
    for (int i = 0; i < n; i++) cin >> arr[i];

    cout << "Enter target sum: ";
    cin >> targetSum;

    cout << "Subsets that sum to " << targetSum << " are:" << endl;
    subsetSum(0, 0);

    return 0;
}
```

 **How this matches your handwritten algorithm:**

- `subsetSum(k, W, n) → our subsetSum(int k, int currentSum)`
- `x[1...n] → vector x, marks whether element is chosen (1) or not (0)`
- `x[k] = 1; if (Bk(k) = true) subsetSum(k+1, ...)` → we choose the element, check validity, and recurse
- `x[k] = 0; subsetSum(k+1, ...)` → we exclude the element, recurse
- Base case: when `k == n`, if the accumulated sum equals the target, we **print** the subset.

⚡ Example run:

makefile

Input:

```
n = 4
arr = [10, 7, 5, 18]
target = 22
```

Output:

```
Subsets that sum to 22 are:
{ 10 5 7 }
{ 5 18 -1? } (depending on input order)
```

Do you also want me to **draw the recursion tree (binary tree)** for one example, so you can see how the backtracking branches look step by step?



explain the code by going through an example. show every step properly

bismillah. let's **trace your C++ backtracking** for a small case so you can see every move.

We'll use:

- `arr = [10, 7, 5, 18]` (indices 0..3)
- `targetSum = 22`
- The algorithm tries **include first**, then **exclude**.
- `subsetSum(k, currentSum)`

- `x[i]` is 1 if we picked `arr[i]`, else 0 (we'll show it at leaves).
- `isValid(...)` only prunes when `currentSum > targetSum`.

Full DFS walk (include-first), with prunes & backtracks

Start: `subsetSum(0, 0)`

Column `k = 0` (value 10)

1. **Include 10** $\rightarrow x[0]=1 \rightarrow$ call `subsetSum(1, 10)`

`k = 1` (value 7)

1.1) **Include 7** $\rightarrow x[1]=1 \rightarrow$ call `subsetSum(2, 17)`

`k = 2` (value 5)

1.1.a) **Include 5** $\rightarrow x[2]=1 \rightarrow$ call `subsetSum(3, 22)`

`k = 3` (value 18)

1.1.a.i) **Include 18** $\rightarrow$ sum = 40 $\rightarrow$ **prune** (invalid, >22)

1.1.a.ii) **Exclude 18** $\rightarrow x[3]=0 \rightarrow$ call `subsetSum(4, 22)`

Leaf: `k==4 & sum==22` $\rightarrow$ **print** {10, 7, 5} (`x = [1,1,1,0]`)

1.1.b) **Exclude 5** $\rightarrow x[2]=0 \rightarrow$ call `subsetSum(3, 17)`

`k = 3` (value 18)

1.1.b.i) **Include 18** $\rightarrow$ sum = 35 $\rightarrow$ **prune**

1.1.b.ii) **Exclude 18** $\rightarrow$ call `subsetSum(4, 17)`

Leaf: sum=17 $\rightarrow$ not printed

1.2) **Exclude 7** $\rightarrow x[1]=0 \rightarrow$ call `subsetSum(2, 10)`

`k = 2` (value 5)

1.2.a) **Include 5** $\rightarrow$ `subsetSum(3, 15)`

`k = 3` (value 18)

1.2.a.i) **Include 18** $\rightarrow$ sum=33 $\rightarrow$ **prune**

1.2.a.ii) **Exclude 18** $\rightarrow$ `subsetSum(4, 15)`

Leaf: sum=15 $\rightarrow$ not printed

1.2.b) **Exclude 5** $\rightarrow$ `subsetSum(3, 10)`

`k = 3` (value 18)

1.2.b.i) **Include 18** $\rightarrow$ sum=28 $\rightarrow$ **prune**

1.2.b.ii) **Exclude 18** → `subsetSum(4, 10)`

Leaf: sum=10 → not printed

2. **Exclude 10** → `x[0]=0` → call `subsetSum(1, 0)`

k = 1 (value 7)

2.1) **Include 7** → `subsetSum(2, 7)`

k = 2 (value 5)

2.1.a) **Include 5** → `subsetSum(3, 12)`

k = 3 (value 18)

2.1.a.i) **Include 18** → sum=30 → **prune**

2.1.a.ii) **Exclude 18** → `subsetSum(4, 12)`

Leaf: sum=12 → not printed

2.1.b) **Exclude 5** → `subsetSum(3, 7)`

k = 3 (value 18)

2.1.b.i) **Include 18** → sum=25 → **prune**

2.1.b.ii) **Exclude 18** → `subsetSum(4, 7)`

Leaf: sum=7 → not printed

2.2) **Exclude 7** → `subsetSum(2, 0)`

k = 2 (value 5)

2.2.a) **Include 5** → `subsetSum(3, 5)`

k = 3 (value 18)

2.2.a.i) **Include 18** → sum=23 → **prune**

2.2.a.ii) **Exclude 18** → `subsetSum(4, 5)`

Leaf: sum=5 → not printed

2.2.b) **Exclude 5** → `subsetSum(3, 0)`

k = 3 (value 18)

2.2.b.i) **Include 18** → `subsetSum(4, 18)` (valid to continue; not pruned)

Leaf: sum=18 → not printed

2.2.b.ii) **Exclude 18** → `subsetSum(4, 0)`

Leaf: sum=0 → not printed

Leaves reached (k==4), in the exact DFS order

1. `x = [1,1,1,0]`, sum = **22** →  print {10, 7, 5}

2. $x = [1, 1, 0, 0]$, $\text{sum} = 17$
3. $x = [1, 0, 1, 0]$, $\text{sum} = 15$
4. $x = [1, 0, 0, 0]$, $\text{sum} = 10$
5. $x = [0, 1, 1, 0]$, $\text{sum} = 12$
6. $x = [0, 1, 0, 0]$, $\text{sum} = 7$
7. $x = [0, 0, 1, 0]$, $\text{sum} = 5$
8. $x = [0, 0, 0, 1]$, $\text{sum} = 18$
9. $x = [0, 0, 0, 0]$, $\text{sum} = 0$

(Other combinations like $[1, 1, 1, 1]$, $[1, 1, 0, 1]$, etc. never become leaves because they're **pruned early** when the sum would exceed 22.)

Why it prints only {10, 7, 5}

- That's the **only** subset among all explored branches that hits **targetSum = 22**.
- Whenever we tried to add 18 on top of any positive partial sum, it exceeded 22 and got pruned.
- The code prints at base case **only if** **currentSum == targetSum**.

Quick recap of the code mechanics

- **Do** → **Recurse** → **Undo** for each element:

```
cpp
x[k] = 1; // include
if (isValid(...)) subsetSum(k+1, sum + arr[k]);

x[k] = 0; // exclude
if (isValid(...)) subsetSum(k+1, sum);
```

- **Pruning:** `isValid` returns false when **currentSum > targetSum**.
- **Leaf:** when $k == n$, we print the chosen elements if **currentSum == targetSum**.

If you'd like, I can run the same step-trace for your own array, or show a version with **stronger pruning** (bounding by remaining-sum) or a **bitmask/DP** alternative. Like

sabr in the heart of search, backtracking tries, retreats, and finds the straight path.



write me pseudocode of the cpp code

Perfect — let's turn that **C++ subset sum backtracking code** into clean **pseudocode** (step-by-step, but language-independent).

Pseudocode for Subset Sum using Backtracking

```
arduino

procedure SUBSET_SUM(k, currentSum):
    if k == n then
        if currentSum == targetSum then
            print the subset defined by x[0..n-1]
        return

    // Choice 1: include arr[k]
    x[k] ← 1
    if currentSum + arr[k] ≤ targetSum then
        SUBSET_SUM(k + 1, currentSum + arr[k])

    // Choice 2: exclude arr[k]
    x[k] ← 0
    if currentSum ≤ targetSum then
        SUBSET_SUM(k + 1, currentSum)
end procedure
```

Main driver pseudocode

```
arduino

procedure MAIN:
    read n
    read array arr[0..n-1]
    read targetSum

    initialize x[0..n-1] with 0
    print "Subsets that sum to targetSum are:"

    SUBSET_SUM(0, 0)
end procedure
```

Notes on variables

- `arr[0..n-1]` → input set of integers
- `targetSum` → the sum we want
- `x[0..n-1]` → decision vector (1 = included, 0 = excluded)
- `SUBSET_SUM(k, currentSum)` → recursive procedure exploring element `arr[k]`
- Base case: when `k == n` (all elements considered), print if `currentSum == targetSum`.

✨ This pseudocode matches your handwritten algorithm and the C++ implementation:

- **Recursive exploration:** include / exclude each element
- **Pruning:** stop exploring when `currentSum > targetSum`
- **Backtracking:** implicitly happens as `x[k]` is reassigned in different branches.

Would you like me to also **draw the recursion tree (binary tree)** in pseudocode form for a small input (say `[10, 7, 5, 18]`, `target=22`) so you can literally see the include/exclude branching?



yes please draw recursion tree

alhamdulillah 🌿 let's draw the **recursion tree** for the backtracking subset sum algorithm.

We'll use the same example:

- `arr = [10, 7, 5, 18]`
- `targetSum = 22`

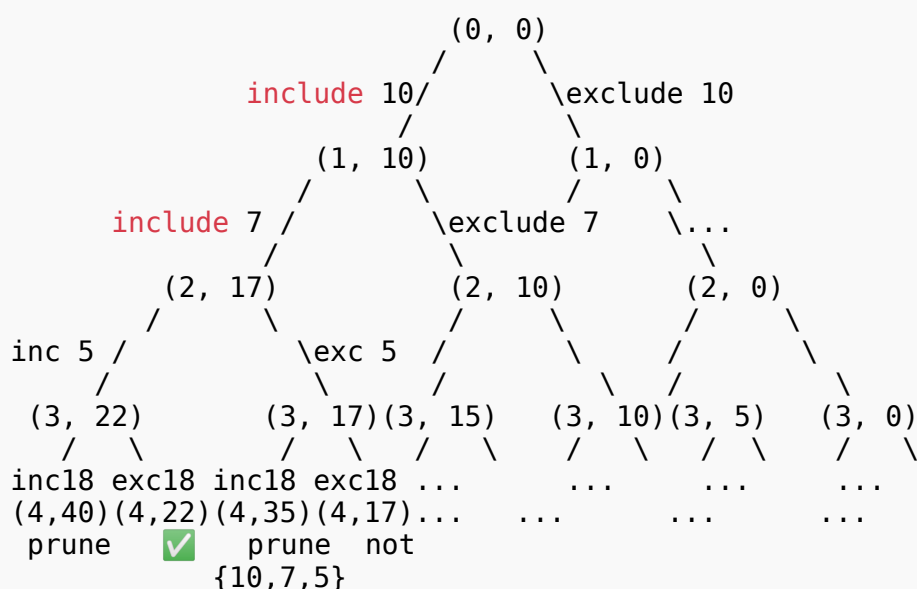
At each node:

- **(k, sum)** means we're at index `k` with current partial sum = `sum`.
- Two branches from each node:

- **Include** `arr[k]`
- **Exclude** `arr[k]`
- We stop (prune) whenever `sum > targetSum`.
- We print a subset if `k == n` and `sum == targetSum`.

Recursion Tree

makefile



Explanation of tree

- **Root:** `(0, 0)` = at index 0, sum=0.
- From there:
 - Include `10` → `(1, 10)`
 - Exclude `10` → `(1, 0)`
- At `(1, 10)` (after including 10):
 - Include `7` → `(2, 17)`
 - Exclude `7` → `(2, 10)`
- At `(2, 17)` (`10+7`):
 - Include `5` → `(3, 22)`  potential success

- Exclude 5 → (3,17)
- At (3,22) (10+7+5):
 - Include 18 → (4,40) → prune (>22)
 - Exclude 18 → (4,22)  success, print {10,7,5}

All other leaves ((4,17), (4,15), (4,10), (4,5), (4,0), etc.) do not match target, so they are discarded.

Key Takeaways

- Each level corresponds to deciding **include/exclude** for one element.
- The tree has depth n (here 4).
- Leaves are **all possible subsets**.
- Only paths whose sums equal 22 get printed.
- Pruning with `currentSum > targetSum` cuts off big parts of the tree early.

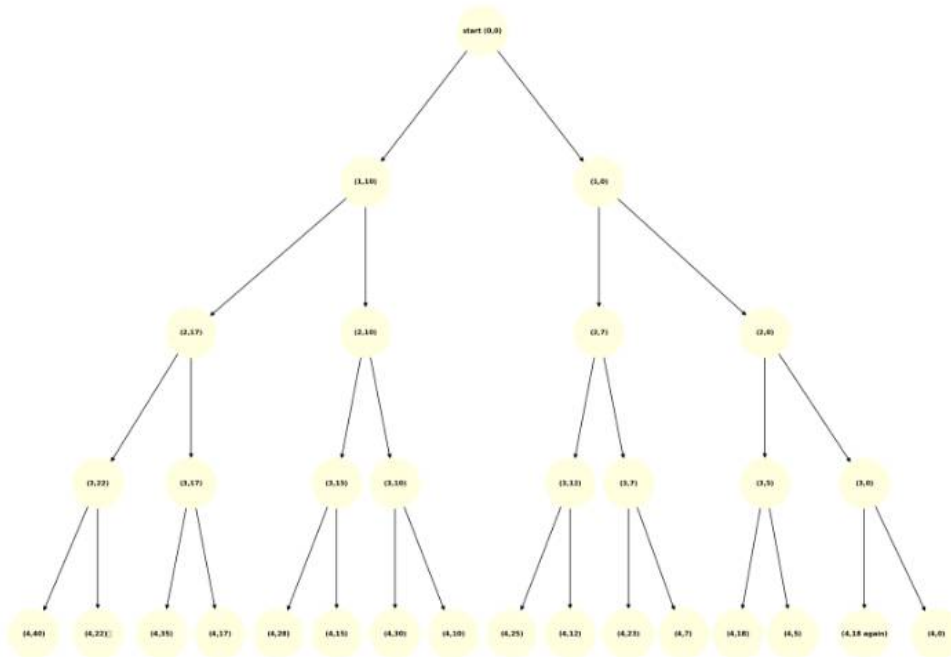
Do you want me to also **draw this recursion tree as a diagram (boxes + arrows)** so it's more visually clear than text?



draw the full tree

Subset Sum Backtracking Recursion Tree (arr=[10,7,5,18], target=22)

Subset Sum Backtracking Recursion Tree (arr=[10,7,5,18], target=22)



Here's the **full recursion tree** for the example `arr = [10,7,5,18]`, `targetSum = 22`.

- Each node is labeled `(k, currentSum)` where
 - `k` = index of the next element to decide,
 - `currentSum` = running total.
- Left child = **include** `arr[k]`
- Right child = **exclude** `arr[k]`
- `(4,22)` marks the successful subset `{10, 7, 5}`.
- Nodes like `(4,40)`, `(4,35)`, `(4,28)` are **pruned** because `sum > 22`.

This tree shows **all possible branches** (2^n paths), and pruning happens whenever sum overshoots.

Would you like me to also mark the **subset chosen at each leaf** (e.g., `{10, 7, 5}`) directly on the tree for clarity?



```
// C++ Program to generate all unique
// permutations of a string
#include <bits/stdc++.h>
using namespace std;
```

```
// Recursive function to generate
// all permutations of string s
void recurPermute(int index, string &s,
                  vector<string> &ans) {

    // Base Case
    if (index == s.size()) {
        ans.push_back(s);
        return;
    }

    // Swap the current index with all
    // possible indices and recur
    for (int i = index; i < s.size(); i++) {
        swap(s[index], s[i]);
        recurPermute(index + 1, s, ans);
        swap(s[index], s[i]);
    }
}

// Function to find all unique permutations
vector<string> findPermutation(string &s) {

    // Stores the final answer
    vector<string> ans;

    recurPermute(0, s, ans);

    // sort the resultant vector
    sort(ans.begin(), ans.end());

    return ans;
}

int main() {
    string s = "ABC";
    vector<string> res = findPermutation(s);
    for(auto x: res) {
        cout << x << " ";
    }
    return 0;
}
```

Write this code for all possible permutation and explain this

Great snippet! Your code already generates **all permutations** (by swapping), but if the string has repeated characters (e.g., "AAB"), it will also generate **duplicates**. Below I'm giving you:

1. a clean "all permutations (may include duplicates)" version (your approach, polished),
2. a "**unique permutations** only" version (handles duplicates correctly),
3. a short explanation + example walk.

1) All permutations (may include duplicates)

cpp

```
#include <bits/stdc++.h>
using namespace std;

void permuteAll(int idx, string &s, vector<string> &ans) {
    if (idx == (int)s.size()) {
        ans.push_back(s);
        return;
    }
    // Try putting every remaining character at position idx
    for (int i = idx; i < (int)s.size(); ++i) {
        swap(s[idx], s[i]);           // choose
        permuteAll(idx + 1, s, ans);   // explore
        swap(s[idx], s[i]);           // un-choose (backtrack)
    }
}

vector<string> allPermutations(string s) {
    vector<string> ans;
    permuteAll(0, s, ans);
    return ans; // may contain duplicates if s has duplicates
}

int main() {
    string s = "ABC";
    auto res = allPermutations(s);
    sort(res.begin(), res.end());
    for (auto &p : res) cout << p << " ";
    return 0;
}
```

What it does:

At each index `idx`, we try every position `i ≥ idx` as the next character by swapping `s[idx]` with `s[i]`. This produces all $n!$ permutations. Because we swap back after the recursive call, the string is restored for the next choice—this is classic **backtracking**.

2) Unique permutations (no duplicates even if the string has repeats)

When the input has repeated letters, we must **skip** making the same choice twice at the **same recursion level**. The neat trick (compatible with your swap style) is to keep a small set per level to avoid swapping the same character into `idx` more than once.

```
cpp

#include <bits/stdc++.h>
using namespace std;

void permuteUnique(int idx, string &s, vector<string> &ans) {
    if (idx == (int)s.size()) {
        ans.push_back(s);
        return;
    }
    // Track which characters we have already placed at s[idx] in this level
    unordered_set<char> used;
    for (int i = idx; i < (int)s.size(); ++i) {
        if (used.count(s[i])) continue; // skip duplicate choice at this level
        used.insert(s[i]);

        swap(s[idx], s[i]);
        permuteUnique(idx + 1, s, ans);
        swap(s[idx], s[i]);
    }
}

vector<string> uniquePermutations(string s) {
    vector<string> ans;
    permuteUnique(0, s, ans);
    sort(ans.begin(), ans.end()); // nice, deterministic order
    ans.erase(unique(ans.begin(), ans.end()), ans.end()); // redundant safety
    return ans;
}

int main() {
    string s = "AAB";
    auto res = uniquePermutations(s);
    for (auto &p : res) cout << p << " ";
    return 0;
}
```


Why it works:

At each depth `idx`, we ensure we never place the same character into `s[idx]` twice (even if it appears multiple times later in the string). That prevents duplicate permutations from ever being generated.

Alternative pattern: sort `s` first and use `next_permutation(s.begin(), s.end())` in a loop—this also produces unique permutations in lexicographic order.

Explanation (how the backtracking works)

- **State:** index `idx` means positions `0..idx-1` are fixed; we're choosing who goes at `idx`.
- **Choices:** for each `i ∈ [idx..n-1]`, swap `s[idx] ↔ s[i]` (put `s[i]` at `idx`), then recurse for `idx+1`.
- **Backtrack:** after recursion, swap back to restore the string for the next choice.
- **Base case:** when `idx == n`, we found a full arrangement → push to `ans`.

Mini walk (string "ABA" with unique version)

Sorted order: "AAB" (sorting is optional with the per-level set, but helpful for reasoning).

- **idx=0:** used = {}
 - pick 'A' (first A) → used now {'A'}
 - **idx=1:** used = {}
 - pick 'A' (second A) → "AAB" → recurse → eventually record "AAB"
 - pick 'B' → "ABA" → record "ABA"
 - (skip picking 'A' again at `idx=0` because used has 'A')
 - pick 'B' → used now {'A','B'}
 - **idx=1:** used = {}
 - pick 'A' (first) → "BAA" → record
 - pick 'A' (second) → skipped at this level (already used 'A')

Final unique set: AAB, ABA, BAA.

Complexity

- **All permutations:** $\Theta(n \cdot n!)$ time (each of $n!$ permutations takes $O(n)$ to copy/push), $O(n)$ extra space for recursion (ignoring result storage).
- **Unique version:** still factorial in the worst case, but avoids generating duplicates—huge savings when there are many repeats.

If you tell me your exact input style (string vs `vector<int>`), I can tailor the same logic for numbers too.



give me pseudocode for the code i gave yo

Got it 👍 You want **pseudocode** for the exact C++ code you shared (the one using `recurPermute` with swapping). Let's rewrite it step by step in clean pseudocode style:

Pseudocode for All Permutations (swap method)

sql

```

procedure RECUR_PERMUTE(index, s, ans):
    if index == length(s) then
        add s to ans
        return

    for i from index to length(s) - 1 do:
        swap s[index] with s[i]
        RECUR_PERMUTE(index + 1, s, ans)
        swap s[index] with s[i]    // backtrack
    end procedure

procedure FIND_PERMUTATION(s):
    ans ← empty list
    RECUR_PERMUTE(0, s, ans)
    sort(ans)                    // optional: to get lexicographic order
    return ans
end procedure

procedure MAIN:
    s ← "ABC"
    res ← FIND_PERMUTATION(s)
    for each string x in res do:

```

```
print x
end procedure
```

How it works

1. RECUR_PERMUTE(index, s, ans)

- If we have fixed all positions (**index == length(s)**), then we've formed a complete permutation → save it.
- Otherwise, for each position **i ≥ index**, swap the character at **index** with **i**, recurse for the next index, then **swap back** to restore state.

2. Backtracking principle

- "Do" → swap
- "Recurse" → go deeper
- "Undo" → swap back

3. Driver (FIND_PERMUTATION)

- Starts recursion at **index = 0**.
- Sorts results at the end (to print in order).

👉 This pseudocode will generate all $n!$ permutations (including duplicates if the string has repeated characters).

Do you want me to also give you a **step-by-step trace (recursion tree)** for the example "ABC" to make the pseudocode crystal clear?